



Architecture

Concepts

Architecture

This guide covers the architectural principles and structure of NautilusTrader:

- Design philosophy and quality attributes.
- Core components and how they interact.
- Environment contexts (backtest, sandbox, live).
- Framework organization and code structure.



Throughout the documentation, the term "*Nautilus system boundary*" refers to operations within the runtime of a single Nautilus node (also known as a "trader instance").

Design philosophy

The major architectural techniques and design patterns employed by NautilusTrader are:

- Domain driven design (DDD)
- Event-driven architecture
- Messaging patterns (Pub/Sub, Req/Rep, point-to-point)
- Ports and adapters
- Crash-only design

These techniques help achieve certain architectural quality attributes.

Quality attributes

Architectural decisions are often a trade-off between competing priorities. The following quality attributes guide design and architectural decisions, roughly in order of weighting.

- Reliability
- Performance
- Modularity
- Testability
- Maintainability
- Deployability

Assurance-driven engineering

NautilusTrader is incrementally adopting a high-assurance mindset: critical code paths should carry executable invariants that verify behaviour matches the business requirements. Practically this means we:

- Identify the components whose failure has the highest blast radius (core domain types, risk and execution flows) and write down their invariants in plain language.
- Codify those invariants as executable checks (unit tests, property tests, fuzzers, static assertions) that run in CI, keeping the feedback loop light.
- Prefer zero-cost safety techniques built into Rust (ownership, `Result` surfaces, `panic = abort`) and add targeted formal tools only where they pay for themselves.
- Track “assurance debt” alongside feature work so new integrations extend the safety net rather than bypass it.

This approach preserves the platform’s delivery cadence while giving high-stakes flows the additional scrutiny they need.

Further reading: [High Assurance Rust](#).

Crash-only design

NautilusTrader draws inspiration from [crash-only design](#) principles, particularly for handling unrecoverable faults. The core insight is that systems which can recover cleanly from crashes

are more robust than those with separate (and rarely tested) graceful shutdown paths.

Key principles:

- Unified recovery path - Startup and crash recovery share the same code path, ensuring it is well-tested.
- Externalized state - Critical state is meant to be persisted externally when configured, reducing data-loss risk; durability depends on the backing store.
- Fast restart - The system is designed to restart quickly after a crash, minimizing downtime.
- Idempotent operations - Operations are designed to be safely retried after restart.
- Fail-fast for unrecoverable errors - Data corruption or invariant violations trigger immediate termination rather than attempting to continue in a compromised state.

 The system does provide graceful shutdown flows (`stop`, `dispose`) for normal operation. These tear down clients, persist state, and flush writers. The crash-only philosophy applies specifically to *unrecoverable faults* where attempting graceful cleanup could cause further damage.

This design complements the fail-fast policy, where unrecoverable errors result in immediate process termination.

References:

- Crash-Only Software - Candea & Fox, HotOS 2003 (original research paper)
- Microreboot: A technique for cheap recovery - Candea et al., OSDI 2004
- The properties of crash-only software - Marc Brooker's blog
- Crash-only software: More than meets the eye - LWN.net article
- Recovery-Oriented Computing (ROC) Project - UC Berkeley/Stanford research

Data integrity and fail-fast policy

NautilusTrader prioritizes data integrity over availability for trading operations. The system employs a strict fail-fast policy for arithmetic operations and data handling to prevent silent data corruption that could lead to incorrect trading decisions.

Fail-fast principles

The system will fail fast (panic or return an error) when encountering:

- Arithmetic overflow or underflow in operations on timestamps, prices, or quantities that exceed valid ranges.
- Invalid data during deserialization including NaN, Infinity, or out-of-range values in market data or configuration.
- Type conversion failures such as negative values where only positive values are valid (timestamps, quantities).
- Malformed input parsing for prices, timestamps, or precision values.

Rationale:

In trading systems, corrupt data is worse than no data. A single incorrect price, timestamp, or quantity can cascade through the system, resulting in:

- Incorrect position sizing or risk calculations.
- Orders placed at wrong prices.
- Backtests producing misleading results.
- Silent financial losses.

By crashing immediately on invalid data, NautilusTrader aims to provide:

1. No silent corruption - The fail-fast policy is intended to prevent invalid data from propagating; this relies on checks covering the inputs.
2. Immediate feedback - Issues are discovered during development and testing, not in production.
3. Audit trail - Crash logs clearly identify the source of invalid data.
4. Deterministic behavior - With deterministic ordering and configuration, the same invalid input should trigger the same failure; nondeterministic sources can vary outcomes.

When fail-fast applies

Panics are used for:

- Programmer errors (logic bugs, incorrect API usage).
- Data that violates fundamental invariants (negative timestamps, NaN prices).
- Arithmetic that would silently produce incorrect results.

Results or Options are used for:

- Expected runtime failures (network errors, file I/O).
- Business logic validation (order constraints, risk limits).
- User input validation.
- Library APIs exposed to downstream crates where callers need explicit error handling without relying on panics for control flow.

Example scenarios

```
// CORRECT: Panics on overflow - prevents data corruption
let total_ns = timestamp1 + timestamp2; // Panics if result > u64::MAX

// CORRECT: Rejects NaN during deserialization
let price = serde_json::from_str("NaN"); // Error: "must be finite"

// CORRECT: Explicit overflow handling when needed
let total_ns = timestamp1.checked_add(timestamp2)?; // Returns Option<UnixNanos>
```

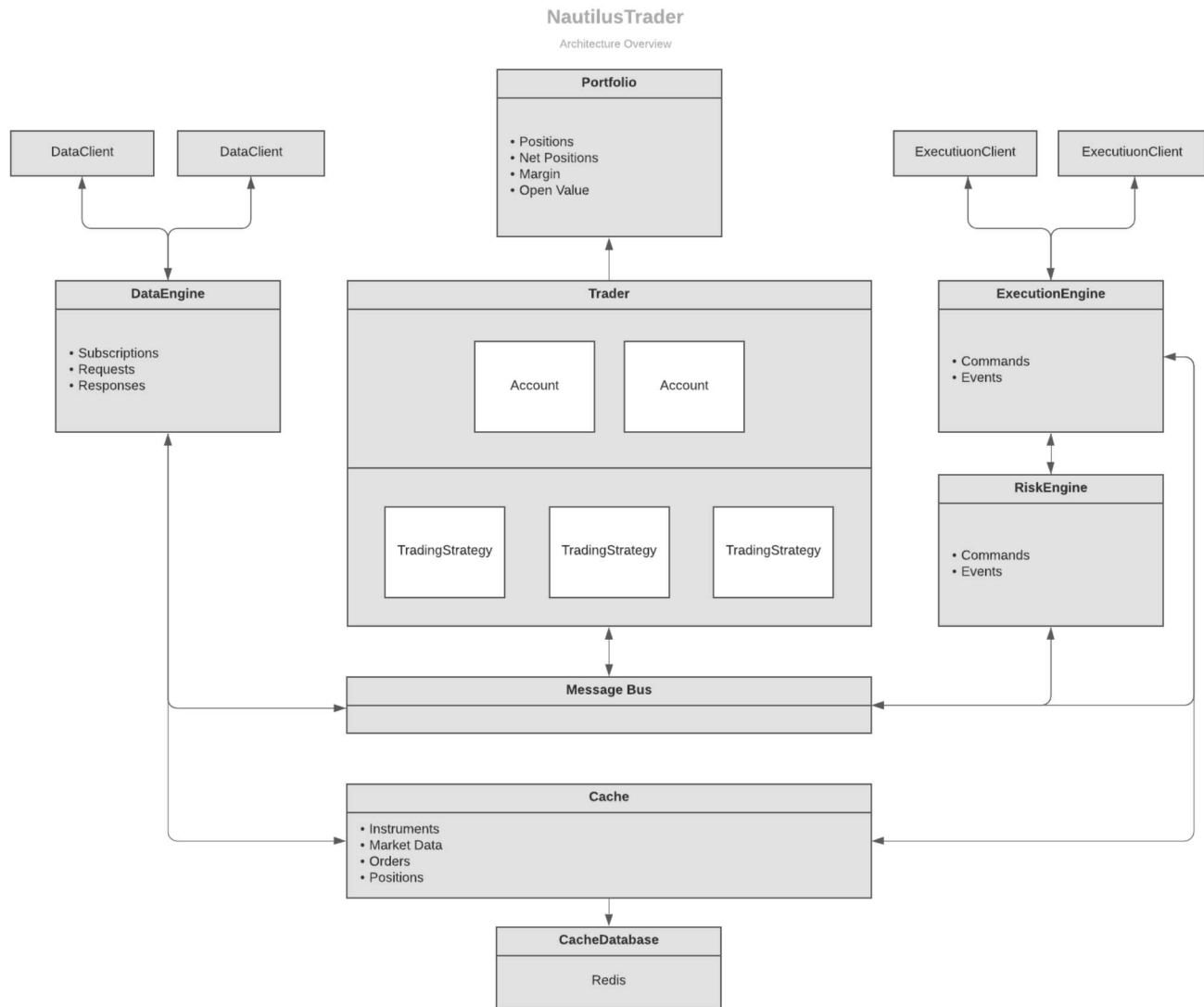


This policy is implemented throughout the core types (`UnixNanos`, `Price`, `Quantity`, etc.) and helps NautilusTrader maintain strong data correctness for production trading.

In production deployments, the system is typically configured with `panic = abort` in release builds, ensuring that any panic results in a clean process termination that can be handled by process supervisors or orchestration systems. This aligns with the crash-only design principle, where unrecoverable errors lead to immediate restart rather than attempting to continue in a potentially corrupted state.

System architecture

The NautilusTrader codebase is actually both a framework for composing trading systems, and a set of default system implementations which can operate in various environment contexts.



Core components

Several core components work together to form the trading system:

NautilusKernel

The central orchestration component responsible for:

- Initializing and managing all system components.
- Configuring the messaging infrastructure.
- Maintaining environment-specific behaviors.
- Coordinating shared resources and lifecycle management.
- Providing a unified entry point for system operations.

MessageBus

The backbone of inter-component communication, implementing:

- Publish/Subscribe patterns: For broadcasting events and data to multiple consumers.
- Request/Response communication: For operations requiring acknowledgment.
- Command/Event messaging: For triggering actions and notifying state changes.
- Optional state persistence: Using Redis for durability and restart capabilities.

Cache

High-performance in-memory storage system that:

- Stores instruments, accounts, orders, positions, and more.
- Provides performant fetching capabilities for trading components.
- Maintains consistent state across the system.
- Supports both read and write operations with optimized access patterns.

DataEngine

Processes and routes market data throughout the system:

- Handles multiple data types (quotes, trades, bars, order books, custom data, and more).
- Routes data to appropriate consumers based on subscriptions.
- Manages data flow from external sources to internal components.

ExecutionEngine

Manages order lifecycle and execution:

- Routes trading commands to the appropriate adapter clients.
- Tracks order and position states.
- Coordinates with risk management systems.
- Handles execution reports and fills from venues.
- Handles reconciliation of external execution state.

RiskEngine

Provides risk management:

- Pre-trade risk checks and validation.
- Position and exposure monitoring.
- Real-time risk calculations.
- Configurable risk rules and limits.

Environment contexts

An environment context in NautilusTrader defines the type of data and trading venue you work with. Understanding these contexts matters for backtesting, development, and live trading.

Here are the available environments you can work with:

- `Backtest`: Historical data with simulated venues.
- `Sandbox`: Real-time data with simulated venues.
- `Live`: Real-time data with live venues (paper trading or real accounts).

Common core

The platform has been designed to share as much common code between backtest, sandbox and live trading systems as possible. This is formalized in the `system` subpackage, where you will find the `NautilusKernel` class, providing a common core system 'kernel'.

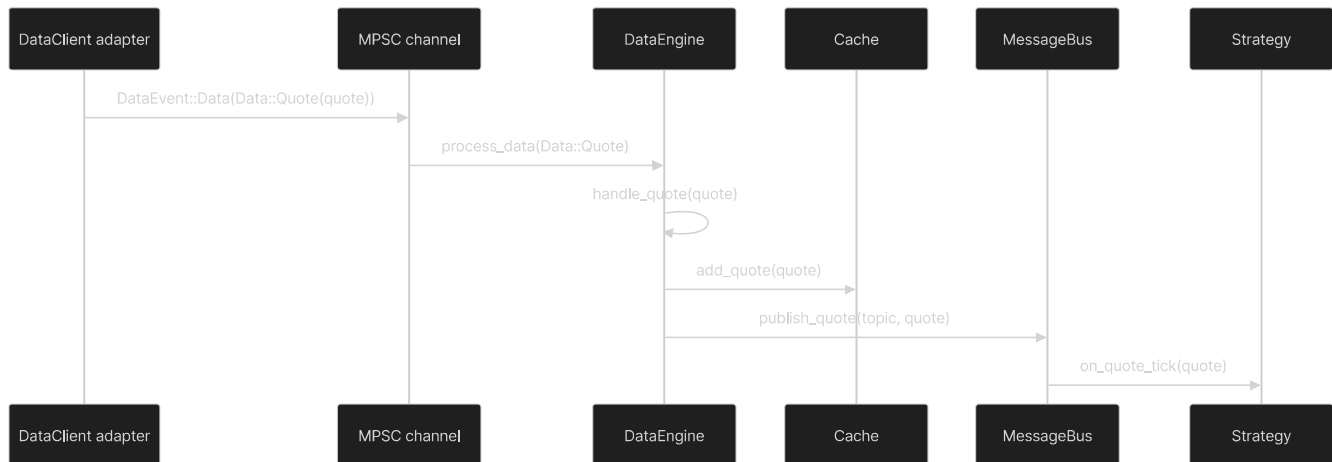
The *ports and adapters* architectural style enables modular components to be integrated into the core system, providing various hooks for user-defined or custom component implementations.

Data and execution flow patterns

Understanding how data and execution flow through the system helps when working with the platform.

Data flow: life of a quote tick

The following trace shows every step a `QuoteTick` takes from the network to your strategy. Trades and bars follow the same cache-then-publish path with different handler names. Order book deltas and depth snapshots take a different route (see the tip below the steps).



Step by step:

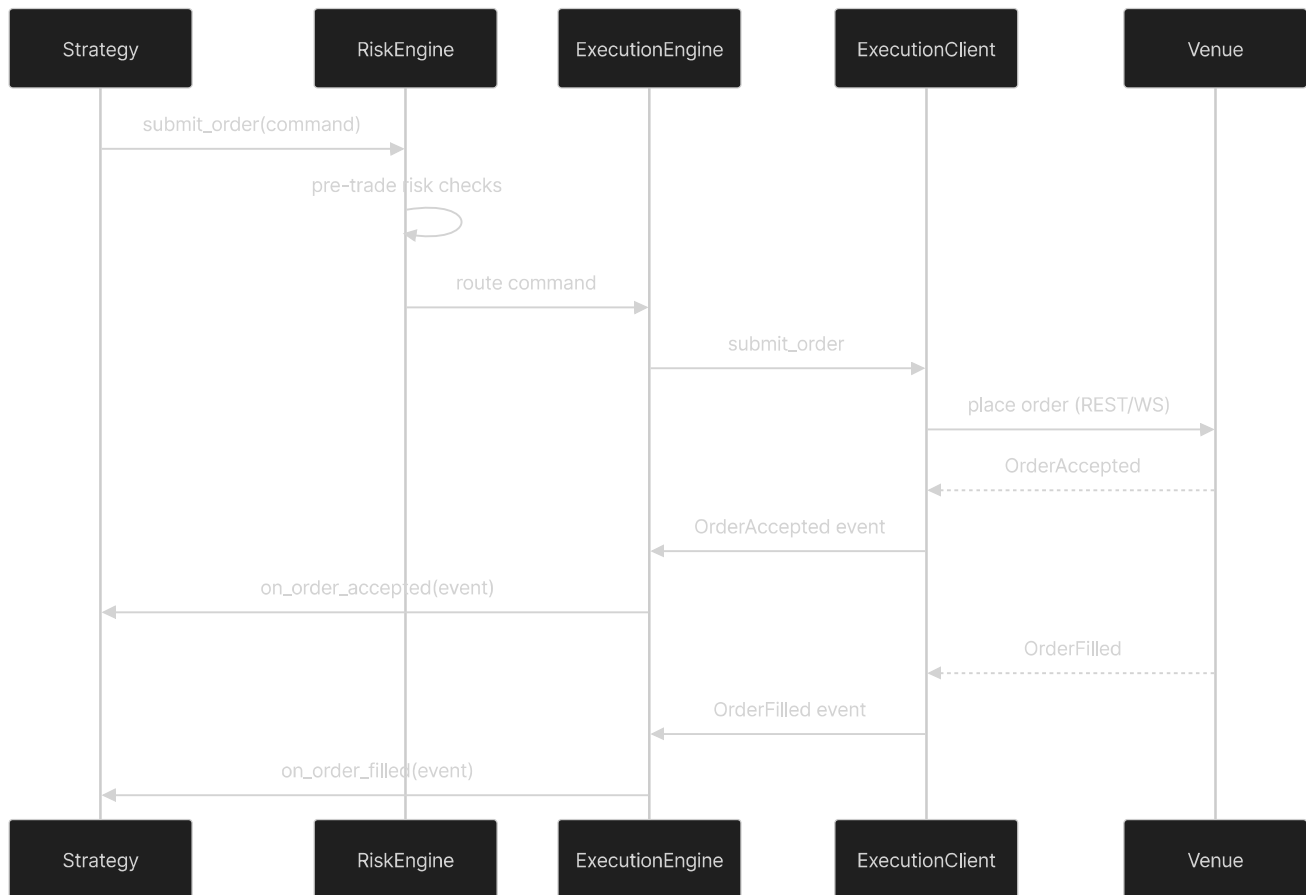
1. Adapter receives raw data. A venue-specific `DataClient` (e.g. Binance, Bybit) receives a WebSocket message, parses it, and constructs a `QuoteTick`.
2. Adapter sends a data event. The adapter sends `DataEvent::Data(Data::Quote.quote))` through an MPSC channel. In live mode this is an async unbounded channel; in backtests the engine feeds data directly.
3. DataEngine processes the event. The channel receiver routes the event to `DataEngine::process_data`, which dispatches to `handle_quote`.
4. Cache stores the quote. `handle_quote` writes the quote into the `Cache` via `cache.add_quote(quote)`, making it available to any component through `self.cache.quote_tick(instrument_id)`.
5. MessageBus publishes. The engine publishes the quote on a topic derived from the instrument ID (e.g. `data.quotes.BINANCE.BTCUSDT-PERP`). The `MessageBus` finds all handlers subscribed to that topic.
6. Strategy handler fires. Each subscribed strategy's `on_quote_tick(quote)` runs on the single-threaded kernel. The quote is already in the cache before the handler executes, so `self.cache.quote_tick(instrument_id)` returns the same quote.



For quotes, trades, and bars the cache-then-publish order means your strategy handler can always read the latest value from the cache. Order book deltas and depth snapshots are published directly; book state is maintained separately through `BookUpdater` subscriptions.

Execution flow: life of an order

When a strategy submits an order, it flows through validation, routing, and back again as execution events:

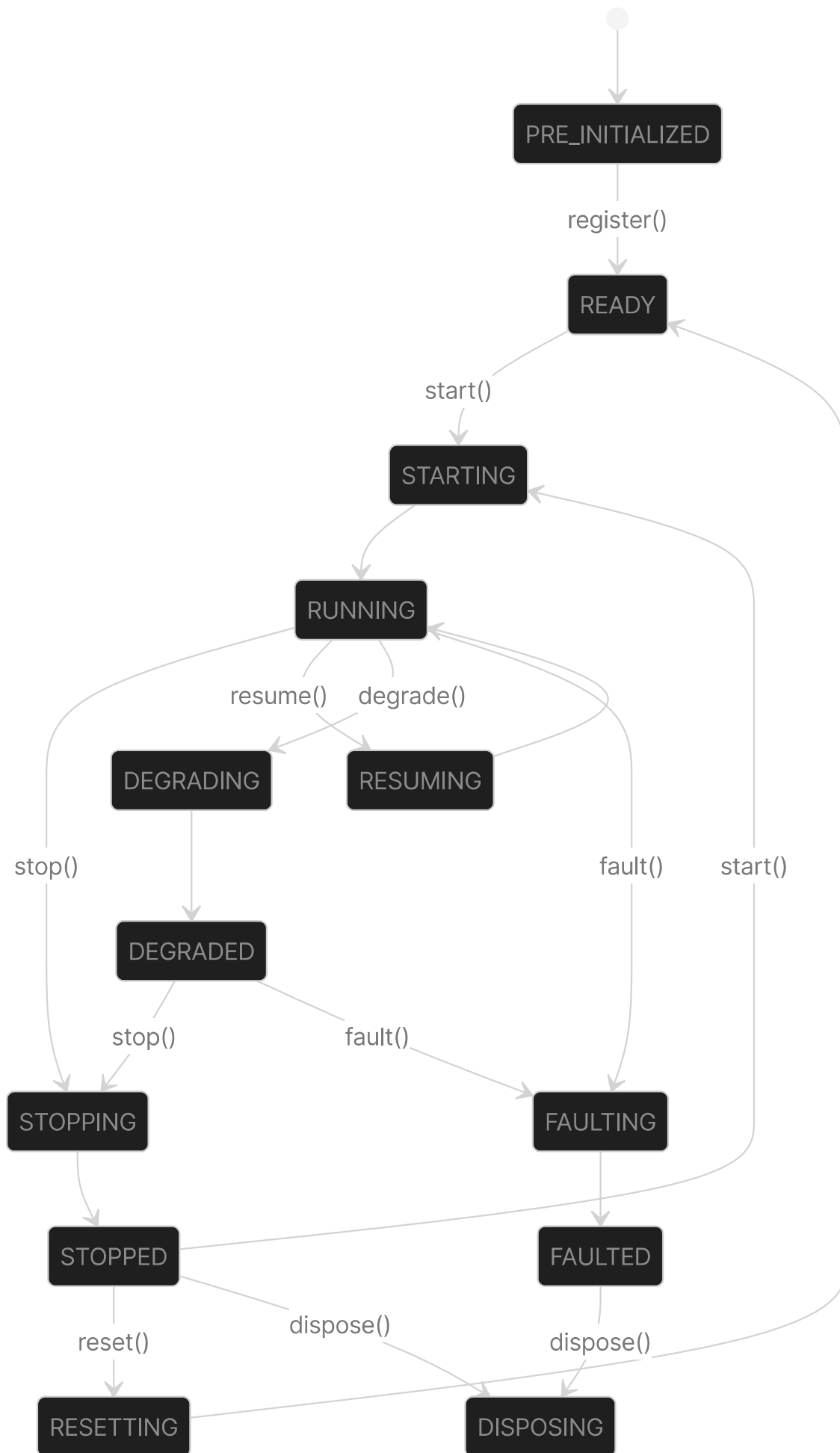


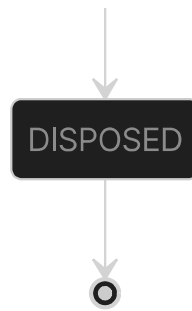
1. Strategy creates a command. The strategy calls `self.submit_order(order)`.
2. RiskEngine validates. Pre-trade checks run (position limits, notional limits, order rate). If a check fails the strategy receives `OrderDenied` and the order never reaches the venue.
3. ExecutionEngine routes. The command is routed to the `ExecutionClient` for the target venue.
4. ExecutionClient submits. The adapter sends the order to the venue over REST or WebSocket.
5. Events flow back. The venue responds with acknowledgments and fills. Each event (Accepted, Filled, Canceled, Rejected, Expired) flows back through the `ExecutionEngine`,

which updates order state in the `Cache` and delivers the event to the strategy's handler. Fill events also trigger position and portfolio updates.

Component state management

All components follow a finite state machine pattern. The `ComponentState` enum defines both stable states and transitional states:





Stable states:

- **PRE_INITIALIZED:** Component is instantiated but not yet ready to fulfill its specification.
- **READY:** Component is configured and able to be started.
- **RUNNING:** Component is operating normally and can fulfill its specification.
- **STOPPED:** Component has successfully stopped.
- **DEGRADED:** Component has degraded and may not meet its full specification.
- **FAULTED:** Component has shut down due to a detected fault.
- **DISPOSED:** Component has shut down and released all of its resources.

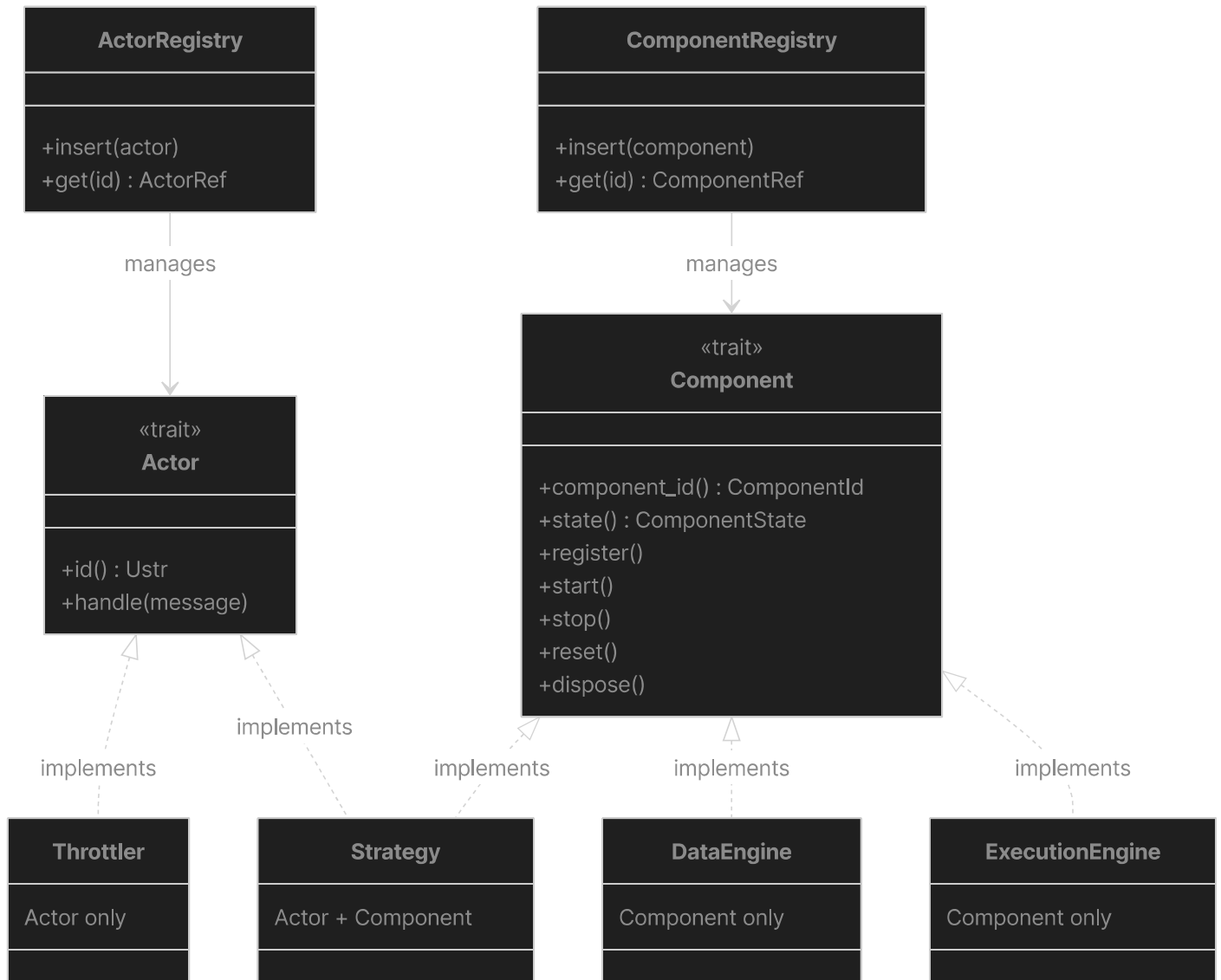
Transitional states:

- **STARTING:** Component is executing its actions on `start`.
- **STOPPING:** Component is executing its actions on `stop`.
- **RESUMING:** Component is being started again after its initial start.
- **RESETTING:** Component is executing its actions on `reset`.
- **DISPOSING:** Component is executing its actions on `dispose`.
- **DEGRADING:** Component is executing its actions on `degrade`.
- **FAULTING:** Component is executing its actions on `fault`.

Transitional states are brief intermediate states that occur during state transitions. Components should not remain in transitional states for extended periods.

Actor vs Component traits

At the Rust implementation level, the system distinguishes between two complementary traits:



Actor trait - Message dispatch:

- Provides the **handle** method for receiving messages dispatched through the actor registry.
- Enables type-safe lookup and message dispatch by actor ID.
- Used by components that need to receive targeted messages (strategies, throttlers).

Component trait - Lifecycle management:

- Manages state transitions (**start**, **stop**, **reset**, **dispose**).
- Provides registration with the system kernel (**register**).
- Tracks component state via the finite state machine described above.
- Used by all system components that need lifecycle management.

i All components can publish and subscribe to messages via the `MessageBus` directly - this is independent of the `Actor` trait. The `Actor` trait specifically enables the registry-based message dispatch pattern where messages are routed to a specific actor by ID.

This separation allows:

- **Actor-only:** Lightweight message handlers without lifecycle (e.g., `Throttler`).
- **Component-only:** System infrastructure with lifecycle but using direct `MessageBus` pub/sub (e.g., `DataEngine`, `ExecutionEngine`).
- **Both traits:** Trading strategies that need lifecycle management AND targeted message dispatch.

The traits are managed by separate registries to support their different access patterns - lifecycle methods are called sequentially, while message handlers may be invoked re-entrantly during callbacks.

Messaging

For modularity and loose coupling, an efficient `MessageBus` passes messages (data, commands, and events) between components.

Threading model

Within a node, the *kernel* consumes and dispatches messages on a single thread. The kernel encompasses:

- The `MessageBus` and actor callback dispatch.
- Strategy logic and order management.
- Risk engine checks and execution coordination.
- Cache reads and writes.

This single-threaded core provides deterministic event ordering and helps maintain backtest-live parity, though live inputs and latency can still cause behavioral differences. Components consume messages synchronously in a pattern *similar* to the actor model.

 Of interest is the LMAX exchange architecture, which achieves award winning performance running on a single thread. You can read about their *disruptor* pattern based architecture in [this interesting article](#) by Martin Fowler.

Background services use separate threads or async runtimes:

- Network I/O - WebSocket connections, REST clients, and async data feeds.
- Persistence - DataFusion queries and database operations via multi-threaded Tokio runtime.
- Adapters - Async adapter operations via thread pool executors.

These services communicate results back to the kernel via the `MessageBus`. The bus itself is thread-local, so each thread has its own instance, with cross-thread communication occurring through channels that ultimately deliver events to the single-threaded core.

Framework organization

The codebase organizes into layers of abstraction, grouped into logical subpackages of cohesive concepts. You can navigate to the documentation for each subpackage from the left nav menu.

Core / low-level

- `core`: Constants, functions and low-level components used throughout the framework.
- `common`: Common parts for assembling the frameworks various components.
- `network`: Low-level base components for networking clients.
- `serialization`: Serialization base components and serializer implementations.
- `model`: Defines a rich trading domain model.

Components

- `accounting`: Different account types and account management machinery.
- `adapters`: Integration adapters for the platform including brokers and exchanges.
- `analysis`: Components relating to trading performance statistics and analysis.

- `cache`: Provides common caching infrastructure.
- `data`: The data stack and data tooling for the platform.
- `execution`: The execution stack for the platform.
- `indicators`: A set of efficient indicators and analyzers.
- `persistence`: Data storage, cataloging and retrieval, mainly to support backtesting.
- `portfolio`: Portfolio management functionality.
- `risk`: Risk specific components and tooling.
- `trading`: Trading domain specific components and tooling.

System implementations

- `backtest`: Backtesting componentry as well as a backtest engine and node implementations.
- `live`: Live engine and client implementations as well as a node for live trading.
- `system`: The core system kernel common between `backtest`, `sandbox`, `live` environment contexts.

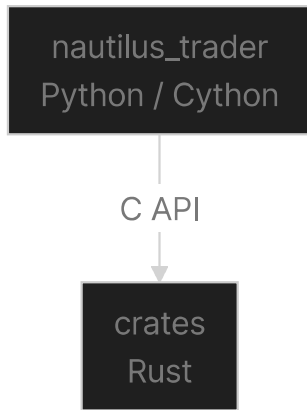
Code structure

The foundation of the codebase is the `crates` directory, containing a collection of Rust crates including a C foreign function interface (FFI) generated by `cbindgen`.

The bulk of the production code resides in the `nautilus_trader` directory, which contains a collection of Python/Cython subpackages and modules.

Python bindings for the Rust core are provided by statically linking the Rust libraries to the C extension modules generated by Cython at compile time (effectively extending the CPython API).

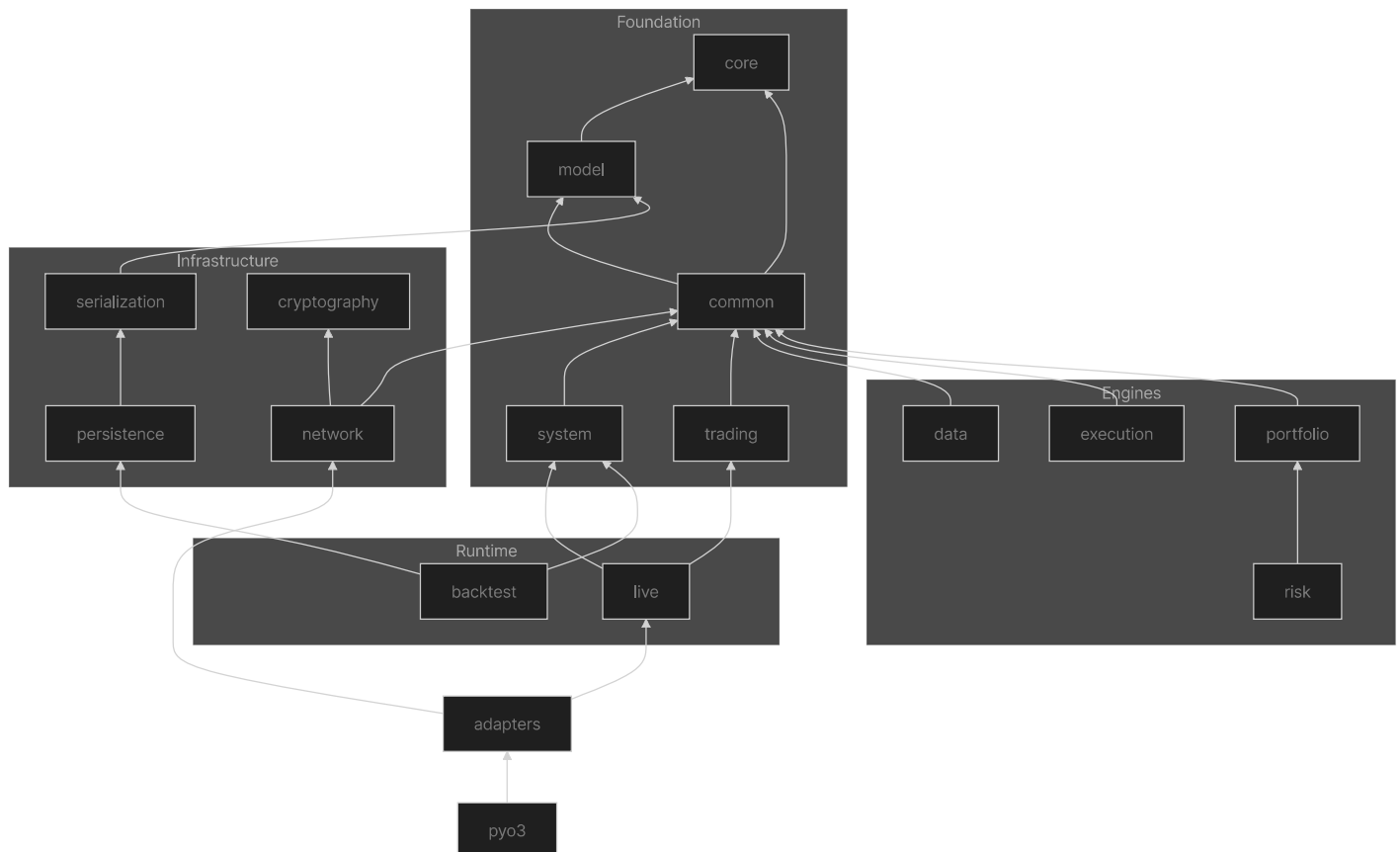
Dependency flow



Rust crates

The `crates/` directory contains the Rust implementation organized into focused crates with clear dependency boundaries. Feature flags control optional functionality - for example, `streaming` enables persistence for catalog-based data streaming, and `cloud` enables cloud storage backends (S3, Azure, GCP).

Dependency flow (arrows point to dependencies):



Crate categories:

Category	Crates	Purpose
Foundation	<code>core</code> , <code>model</code> , <code>common</code> , <code>system</code> , <code>trading</code>	Primitives, domain model, kernel, actor & strategy base.
Engines	<code>data</code> , <code>execution</code> , <code>portfolio</code> , <code>risk</code>	Core trading engine components.
Infrastructure	<code>serialization</code> , <code>network</code> , <code>cryptography</code> , <code>persistence</code>	Encoding, networking, signing, storage.
Runtime	<code>live</code> , <code>backtest</code>	Environment-specific node implementations.
External	<code>adapters/*</code>	Venue and data integrations.
Bindings	<code>pyo3</code>	Python bindings.

Feature flags:

Feature	Crates	Effect
<code>streaming</code>	<code>data</code> , <code>system</code> , <code>live</code>	Enables <code>persistence</code> dependency for catalog streaming.
<code>cloud</code>	<code>persistence</code>	Enables cloud storage backends (S3, Azure, GCP, HTTP).
<code>python</code>	most crates	Enables PyO3 bindings (auto-enables <code>streaming</code> , <code>cloud</code>).
<code>defi</code>	<code>common</code> , <code>model</code> , <code>data</code>	Enables DeFi/blockchain data types.



Both Rust and Cython are build dependencies. The binary wheels produced from a build do not require Rust or Cython to be installed at runtime.

Type safety

The platform design prioritizes software correctness and safety.

The Rust codebase under `crates/` relies on the `rustc` compiler's guarantees for safe code. Any `unsafe` blocks are explicit opt-outs where we must uphold the required invariants ourselves (see

the Rust section of the [Developer Guide](#)); overall memory and type safety depend on those invariants holding.

Cython provides type safety at the C level at both compile time, and runtime:

 If you pass an argument with an invalid type to a Cython implemented module with typed parameters, then you will receive a `TypeError` at runtime.

If a function or method's parameter is not explicitly typed to accept `None`, passing `None` as an argument will result in a `ValueError` at runtime.

 The above exceptions are not explicitly documented to prevent excessive bloating of the docstrings.

Errors and exceptions

The documentation aims to cover all possible exceptions that NautilusTrader code can raise, and the conditions that trigger them.

 There may be other undocumented exceptions which can be raised by Python's standard library, or from third party library dependencies.

Processes and threads

One node per process

Running multiple `TradingNode` or `BacktestNode` instances concurrently in the same process is not supported due to global singleton state:

- Backtest force-stop flag - The `_FORCE_STOP` global flag is shared across all engines in the process.
- Logger mode and timestamps - The logging subsystem uses global state; backtests flip between static and real-time modes.
- Runtime singletons - Global Tokio runtime, callback registries, and other `OnceLock` instances are process-wide.

Sequential execution of multiple nodes (one after another with proper disposal between runs) is fully supported and used in the test suite.

For production deployments, add multiple strategies to a single `TradingNode` within a process. For parallel execution or workload isolation, run each node in its own separate process.

Related guides

- [Overview](#) - High-level introduction to NautilusTrader.
- [Message Bus](#) - Core messaging infrastructure.

[< Overview](#)[Previous Page](#)[Actors >](#)[Next Page](#)